THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 03 OF 07

Design 2 — Credit Card Authorization System

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

03



YOUR STUDY COMPANION

Module 2: Financial Systems — Deep Dive Focus: Low latency, financial correctness, concurrency, fraud detection, idempotency, high availability, and failure recovery.

> **Scope:** An illustrative **issuer-side** credit card authorization system receiving requests from a payment network. This is an educational design, **not** a description of American Express's internal systems. Capacity numbers and latency targets below are hypothetical.

NAVIGATION

Inside this guide

01 1. Understand the payment lifecycle

02 2. Derive the requirements

03 3. Identify business invariants

04 4. Select the communication protocol

05 5. Begin with the simplest baseline

06 6. Solve concurrent authorizations

07 7. Define the data model

08 8. Make authorization approval atomic

09 9. Handle duplicate requests: idempotency

10 10. Verify card credentials and evaluate fraud

11 11. End-to-end authorization flow

12 12. Derive a latency budget

13 13. Model authorization and hold lifecycles

14 14. Move from authorization to clearing

15 15. Handle failures explicitly

16 16. Where Kafka or a message broker belongs

17 17. High availability and multi-region design

18 18. Scale authorization capacity

19 19. Security architecture

20 20. Observability and operations

21 21. Final reference architecture

22 22. Decisions and rejected alternatives

23 23. Variations that change the architecture

24 24. Summary: Key concepts demonstrated

25 Official references

CHAPTER

1. Understand the payment lifecycle

Before designing the system, separate three operations:

```
Authorization
    |
    v
Clearing / Presentment
    |
    v
Settlement
```

- **Authorization:** The issuer decides whether a transaction can be approved and typically reserves the relevant credit.
- **Clearing/presentment:** A merchant submits the completed transaction for financial posting and processing.
- **Settlement:** The relevant parties complete financial settlement obligations under applicable arrangements.

These stages are distinguished in the American Express U.S. Merchant Reference Guide.

Example: Purchase a $100 item

1. The cardholder presents a card.
2. The merchant requests authorization for $100.
3. The issuer verifies card and account status, available credit, and risk.
4. The issuer approves and places an authorization hold.
5. The response returns to the merchant.
6. Later, the merchant submits the transaction for clearing.
7. The issuer posts the applicable financial obligation.
8. Settlement occurs under the relevant payment arrangements.

Key point: Approval is not the same as transferring $100 to the merchant. Authorization and financial posting are distinct business events.

Payment participants

```
Cardholder
    |
    v
Merchant
    |
    v
Processor / Acquirer
    |
    v
Card Network
    |
    v
Card Issuer
    |
    +-- Card and credential verification
    +-- Fraud / risk decision
    +-- Credit availability check
    +-- Authorization decision
    |
    v
```

```
Approval / Decline
    |
    v
Response back through network
```

Our ownership boundary: The issuer-side system. We do not redesign checkout, acquiring, the card network, or the complete clearing and settlement platform.

CHAPTER

2. Derive the requirements

Functional requirements

1. Receive authorization requests from a trusted card network.
2. Identify the card and account.
3. Validate card status and required authentication information.
4. Enforce account and product policies.
5. Evaluate fraud risk.
6. Check available credit or product-specific spending capacity.
7. Create a hold on approval.
8. Return the correct network response.
9. Recognize repeated logical requests without duplicate holds.
10. Support reversals, adjustments, and expiry.
11. Retain records for audit, reconciliation, and clearing.

Non-functional requirements

Requirement	Priority	Why
Correctness	Critical	Prevent unintended double reservation and overcommitment
Response latency	Critical	Customer and merchant are waiting
Availability	Critical	Outages interrupt purchases
Security	Critical	Sensitive payment credentials and financial data
Consistency	Critical for credit reservation	Concurrent approvals share credit availability
Scale	High	High-volume and bursty transaction traffic
Observability	High	Investigate uncertain outcomes and incidents
Auditability	High	Financial decisions require a traceable history

Hypothetical scale and objectives

Parameter	Illustrative assumption

Authorizations/day	50 million
Average transactions/second	~579 TPS
Peak multiplier	10x
Illustrative peak	~5,800 TPS
Internal issuer-side p99 latency target	Under 300 ms
Internal service availability objective	99.99%
Independent active-active writes for one account	Not required initially

The 300 ms is **not a verified card-network limit**. Actual deadlines come from the applicable network interface agreements. We use these figures to reason about design choices.

CHAPTER

3. Identify business invariants

An **invariant** is a property that must remain true despite concurrency or failures.

1. **No duplicate credit reservation:** A retry of the same logical authorization must not create a second hold.
2. **No unintended credit overcommitment:** Concurrent requests cannot both spend the same remaining credit under the defined policy.
3. **Durable approval:** Before returning a normal approval, persist the decision and reservation durably.
4. **Idempotent reversal:** A duplicate reversal must not release credit twice.
5. **Correct lifecycle:** Authorization, clearing, reversal, and settlement must not be confused.

Exception mechanisms such as network stand-in authorization or issuer-approved credit exceptions require separately defined policies. They are not reasons to relax the normal system's invariants accidentally.

Design implication: Database transactions, unique constraints, request identities, controlled ownership, and reconciliation are more important here than choosing a trendy technology.

CHAPTER

4. Select the communication protocol

The issuer normally receives network financial messages, not an HTTP checkout request directly from the customer.

```
Payment Network
     |
     v
Network Protocol Adapter
     |
     v
Canonical Authorization Request
     |
     v
Authorization Service
```

Choice: A protocol adapter / anti-corruption layer

The adapter handles:

- Secure network connectivity and validation.
- Message parsing and required field validation.
- Correlation and transaction identity.
- Duplicate message recognition.
- Conversion to the internal request model.
- Mapping decisions to network response codes and messages.

Card payment protocols frequently use ISO 8583-based message structures, but the exact interface and field semantics are network-specific. Never assume a generic ISO 8583 message is the complete specification for a particular network.

Concept — Anti-corruption layer: An integration boundary that prevents external protocols, identifiers, and conventions from spreading throughout internal business services.

Example **illustrative internal** request:

```
{
  "requestId": "network-transaction-reference",
  "cardReference": "card_tok_123",
  "merchantId": "merchant_456",
  "amountMinor": 10000,
  "currency": "USD",
  "transactionType": "PURCHASE",
  "channel": "CARD_PRESENT",
  "timestamp": "2026-11-02T12:30:00Z"
}
```

Alternative: Make every internal service understand the network's raw message format. This saves mapping but creates hard dependencies and complicates adding a second network or upgrading a protocol.

Decision: Use a canonical internal model while preserving all network information needed for response, clearing, investigations, and reconciliation.

CHAPTER

5. Begin with the simplest baseline

```
Card Network
     |
     v
Network Adapter
     |
     v
Authorization Service
     |
     +--> Card Database
     +--> Account Database
     +--> Fraud Rules
     |
     v
Approve / Decline
```

The service resolves the card, checks eligibility and risk, checks available credit, records its decision, and sends a response.

This is a functional starting point. Its first serious weakness is concurrency: what if two purchases reach the issuer at nearly the same time?

CHAPTER

6. Solve concurrent authorizations

Assume:

Measure	Amount
Credit limit	$1,000
Posted outstanding	$600
Existing holds	$250
Available credit	**$150**

Simplified revolving-credit formula:

```
Available credit = Credit limit − Posted exposure − Active holds
```

Real products may also consider payments, adjustments, disputes, over-limit rules, or dynamic spending capacity.

Now merchant A requests $100 while merchant B independently requests $100.

Broken read-then-write design

```
Thread A                    Thread B
   |                           |
Read available = $150       Read available = $150
   |                           |
$100 <= $150                $100 <= $150
   |                           |
Approve                     Approve
   |                           |
Save $100 hold              Save $100 hold
```

Both could approve, consuming $200 of the same $150. This is a **race condition**.

Candidate concurrency controls

Option	Mechanism	Trade-off
Pessimistic lock	Lock account row while processing	Simple but can increase contention
Optimistic concurrency	Verify version before updating	Retries if competing updates win
Atomic conditional update	Update only if sufficient availability remains	Short critical section; requires correct counter modeling

Choice: Atomic conditional reservation

Illustrative SQL:

```
UPDATE account_credit
SET reserved_minor = reserved_minor + :amount
WHERE account_id = :account_id
  AND credit_limit_minor
      - posted_exposure_minor
      - reserved_minor >= :amount
RETURNING reserved_minor;
```

Interpretation:

- One row updated: reservation succeeded.
- Zero rows updated: insufficient available credit or no matching account; distinguish the specific reason before returning a decision.
- Related authorization and hold records must commit in the **same database transaction**.

With correct transaction isolation and row-update semantics, concurrent updates on the same account cannot both reserve credit already consumed by the first committed update.

Result: one of the $100 requests can be approved; the other is declined under the simplified policy. The exact winner is nondeterministic.

Why not use Redis as the authoritative balance?

Redis supports atomic operations, but separating the reservation counter from the durable financial database raises difficult questions:

- What happens if Redis accepts a reservation and database persistence fails?
- What durability guarantees survive Redis failover?
- How are reversals, clearing, and reservations reconciled?
- Which system becomes authoritative during recovery?

Decision: Begin with a transactional authoritative database. Use Redis or another low-latency store for eligible risk features and caches, not as an accidentally inconsistent financial truth source.

CHAPTER

7. Define the data model

The design needs account exposure, the authorization decision, the hold, and durable request identity.

`AccountCredit`

```
account_id              PK
credit_limit_minor
posted_exposure_minor
reserved_minor
currency
version
updated_at
```

Authorization

```
authorization_id        PK
request_identity        UNIQUE
account_id
merchant_id
requested_amount_minor
approved_amount_minor
currency
status
authorization_code
created_at
expires_at
version
```

AuthorizationHold

```
hold_id                 PK
authorization_id        UNIQUE
account_id
reserved_amount_minor
remaining_amount_minor
status
created_at
expires_at
```

ProcessedRequest

```
request_identity        PK
request_fingerprint
authorization_id
response_code
response_reference
processing_status
```

Why SQL initially? We need unique constraints, atomic conditional updates, transactional writes across related records, and controlled lifecycle changes. A relational database supports these well.

A distributed transactional NoSQL database could also work if justified by operational and scale needs; SQL is not the only possible correct implementation.

Critical consistency rule

`AccountCredit.reserved_minor` and the detailed active holds must be maintained consistently. Every hold mutation must update the aggregate credit reservation within the **same local transaction**. Run independent reconciliation jobs to find mismatches.

Card credentials and sensitive authentication information should remain in protected card-security systems, not general business tables.

CHAPTER

8. Make authorization approval atomic

Our central write path:

```
BEGIN TRANSACTION

  1. Claim unique request identity.
  2. If the request exists, recover its decision.
  3. Attempt conditional credit reservation.
  4. Create the authorization record.
  5. Create the hold (if approved).
  6. Save decision / response reference.
  7. Save an outbox event.

COMMIT

Return the committed decision.
```

When a request identity already exists, compare the canonical request fingerprint. A repeated key with different logical details is a conflict, not proof that the new request is a legitimate retry.

Concurrent duplicates are coordinated through the unique constraint and appropriate transaction handling; the second may wait or receive a retryable processing outcome.

Why one transaction?

Without atomicity:

```
Reserve $100            SUCCESS
Insert authorization    FAILURE

Result: orphan reservation.
```

With atomicity, failure of a required insert rolls back the reservation.

Concept — ACID atomicity: All operations in one database transaction commit, or none do.

Keep the critical section short

Do not hold database locks while waiting for an external risk service. Prefer:

1. Validate request and card status.
2. Perform credential and required fraud/risk checks.
3. Enter a short database transaction.
4. Recheck critical current state and hard limits.
5. Reserve credit, persist decision, and commit.

Concept — Critical section: A region of execution that must be coordinated to protect shared state. Shorter lock duration reduces contention.

A strict rule that can change between fraud evaluation and commit must be versioned, rechecked, or coordinated in the final decision path.

CHAPTER

9. Handle duplicate requests: idempotency

A response can be lost after a successful commit:

```
Network                        Issuer
   |                             |
   |----- Authorize $100 --------->|
   |                             |
   |                          Commit $100 hold
   |                             |
   |<----- Response lost ----------X
   |
   |----- Retry same request ----->|
```

Naively processing the retry would reserve $200 for the same logical transaction.

Concept — Idempotency: Repeating the same logical operation does not produce another business effect.

Choose a durable request identity

Issuer authorization identity should use the appropriate network-defined transaction references and scoped identifying fields. Amount, timestamp, or one trace number alone may not be globally unique.

```
Incoming Request
       |
       v
Derive Request Identity
       |
       v
Find Previous Decision
       |
       +-- Existing, same request --> Return recorded decision
       |
       +-- Existing, different ----> Identity conflict
       |
       +-- New --------------------> Process once and persist
```

Use a durable unique constraint as the **correctness mechanism**. An optional Redis duplicate lookup can be an optimization.

Important: An incremental authorization, a reversal, and a clearing message are separate logical operations. They must not accidentally share the authorization request's idempotency scope.

CHAPTER

10. Verify card credentials and evaluate fraud

Three questions are distinct:

1. **Card/credential authentication:** Is the payment credential valid in its transaction context?
2. **Fraud risk:** Is this purchase suspicious?
3. **Credit authorization:** Does account policy permit the amount?

Card security service

```
Authorization Service
```

```
          |
          v
Card Security Service
          |
          +--> Protected Card Data
          |
          +--> HSM
```

Concept — HSM: A hardware security module protects cryptographic keys and performs sensitive cryptographic operations within a controlled boundary.

Why use a separate security boundary? General application servers should not have broad access to payment keys and sensitive card data.

Risk decision

Possible inputs:

Signal	Example
Amount	Unusually large purchase
Merchant	Elevated merchant risk
Velocity	Many attempts recently
Geography	Unexpected geographic pattern
Channel	Card-present vs. online
Device / authentication	Authentication confidence
History	Deviation from past spending

```
Authorization Request
         |
         v
      Risk Engine
       /      \
      v        v
  Rules      ML Model
      \        /
       v      v
     Risk Decision
```

Rules enforce explicit conditions, while **ML inference** estimates probabilistic risk. Business policy determines what to do with the outputs.

Why risk assessment is synchronous

If mandatory fraud evaluation runs only after approval through Kafka, the issuer may discover fraud too late. Necessary pre-approval controls belong on the synchronous authorization path.

Online feature store

```
Historical Transactions
          |
          v
    Stream Processor
          |
```

```
        v
  Feature Store
       ^
       |
   Risk Engine
```

Concept — Feature store: A service storing precomputed signals for online scoring, such as recent counts, merchant patterns, and spend statistics.

A low-latency key-value store can be useful. But features and **hard policy constraints** are different:

- A statistical trend may tolerate short staleness.
- A strict transaction-count limit needs coordinated, enforceable updates.

Fraud service outage

Fallback	Benefit	Cost / risk
Fail closed	Avoid required checks being skipped	Declines legitimate purchases
Fail open	Higher acceptance	Increased risk
Degraded rules	Some decisions remain possible	Less sophisticated assessment

Decision: Use explicit risk-approved degradation policies. Mandatory authentication and critical controls cannot be silently bypassed.

CHAPTER

11. End-to-end authorization flow

```
                CARD NETWORK
                     |
                     v
              Network Gateway
                     |
                     v
              Protocol Adapter
                     |
                     v
          Authorization Service
                /       \
               v         v
      Card Security   Risk Engine
          |              |
          v              v
      Card / HSM      Rules / ML
          \              /
           v            v
            Decision Policy
                     |
                     v
              Credit Reservation
                     |
                     v
      +-----------------------+
```

```
| Transactional DB      |
| - Account Credit      |
| - Authorization       |
| - Holds               |
| - Request Identity    |
| - Outbox              |
+-----------------------+
           |
           v
     Commit Decision
           |
           v
     Protocol Adapter
           |
           v
      CARD NETWORK
```

Sequence

Step	Action	Reason
1	Receive network message	Start the operation
2	Validate message and fields	Reject malformed input
3	Derive request identity	Enable deduplication
4	Find previous decision	Avoid duplicate holds
5	Resolve and validate card	Establish eligibility
6	Validate required credentials	Prevent unauthorized use
7	Evaluate risk	Enforce fraud policy
8	Begin short database transaction	Create atomic boundary
9	Revalidate critical current state	Avoid stale decisions
10	Reserve credit	Enforce the account invariant
11	Save authorization, hold, and outbox	Persist state and event intent
12	Commit	Make result durable
13	Map decision to network response	Return approval or decline
14	Publish outbox events asynchronously	Decouple secondary workloads

The diagram represents **logical components**, not a mandatory one-service-per-box microservice deployment.

CHAPTER

12. Derive a latency budget

Hypothetical issuer-side p99 objective: **300 ms**.

Component	Illustrative allocated time
Ingress, routing, parsing	20 ms
Credential verification	40 ms
Risk decision	80 ms
Credit checks and transaction	70 ms
Response formatting and egress	20 ms
Contingency	70 ms
Total budget	**300 ms**

These are planning allocations, not benchmarked latencies. The sum of component p99 measurements is **not** the same thing as end-to-end p99. Measure actual traces and full-path latency distributions.

Reduce latency without compromising correctness

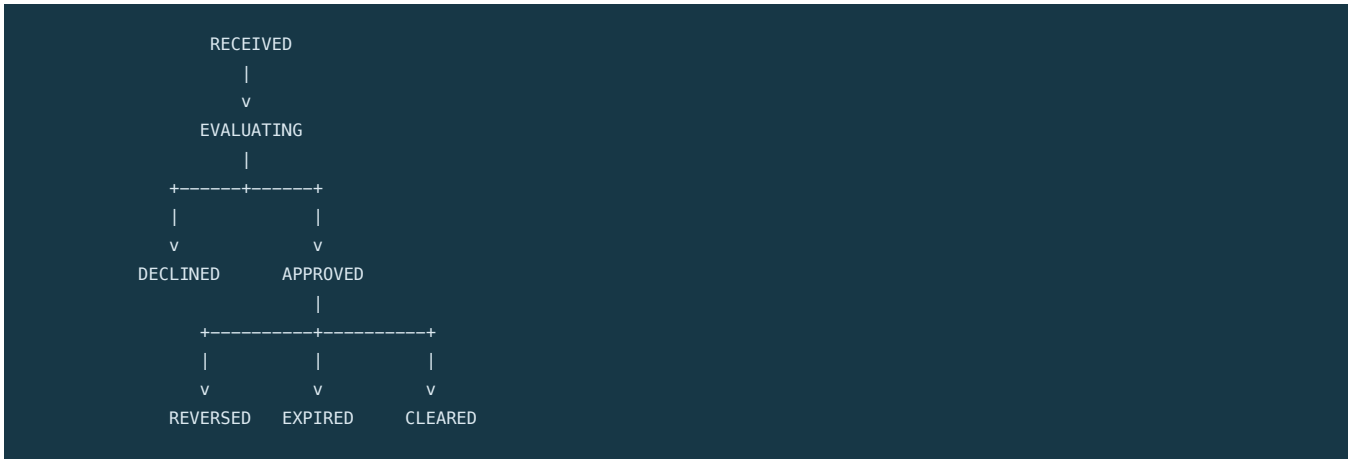
- Precompute risk features that can be safely cached.
- Parallelize independent reads.
- Reuse connections and avoid repeated network handshakes where appropriate.
- Keep database transactions short.
- Avoid analytics or notification calls on the critical path.
- Place latency-sensitive components where network latency is predictable.
- Maintain a bounded failure and retry policy.

Rule: Parallelize independent reads; preserve ordering and atomicity for financial state changes.

CHAPTER

13. Model authorization and hold lifecycles

Simplified logical states:



This is intentionally simplified. Production workflows can involve partial captures, multiple clearing records, incremental authorizations, partial reversals, and other exceptions.

Important modeling choice: Keep the historical *authorization decision* distinct from the *current hold state*. An originally approved authorization remains historically approved even after its hold is released.

Reversal flow

```
Reversal Request
       |
       v
Find Original Authorization
       |
       v
Validate Reversal Identity
       |
       v
Inspect Remaining Hold
       |
       +-- Already applied? --> No second release
       |
       v
Begin Database Transaction
       |
       +--> Release eligible amount
       +--> Update hold lifecycle
       +--> Save reversal record
       +--> Save outbox event
       |
       v
Commit
       |
       v
Return response
```

Why idempotent? Two deliveries of the same $100 reversal cannot each release $100.

For partial reversals, persist reversal identities and amounts so that repeated reversals and genuinely new adjustments are distinguished.

CHAPTER

14. Move from authorization to clearing

Assume the following before clearing:

Before clearing	Amount
Credit limit	$1,000
Posted exposure	$500
Active holds	$100
Available credit	**$400**

The entire $100 authorization then clears and posts:

After clearing	Amount
Credit limit	$1,000
Posted exposure	$600
Active holds	$0
Available credit	**$400**

The exposure moved from *reserved* to *posted*. It did not become a second charge.

Required rule: Clearing must coordinate the posted exposure and the release of the related reservation, so the same obligation is not counted twice.

Different clearing amounts

```
Original authorization:  $100
Final cleared amount:     $95
```

The system must post the actual applicable amount and handle the remaining hold according to its rules. This includes partial capture, multiple presentments, late clearing, and missing authorization matches.

Ledger distinction: An authorization hold is not automatically a double-entry financial journal entry. The financial ledger records accounting events. The hold system manages available-credit reservation. They must be reconciled but should not be conflated.

CHAPTER

15. Handle failures explicitly

Failure A: Crash before database commit

```
Receive request
      |
      v
Evaluate risk
      |
      v
Attempt reservation
      |
      X  Crash before commit
```

Transaction rolls back. No normal approval should have been returned. Retry can safely attempt the logical operation again.

Failure B: Commit succeeds but response is lost

```
Receive request
      |
      v
Commit $100 hold
      |
      v
```

```
Prepare approval
      |
      X  Network failure
```

The financial effect exists, but the merchant may not have received the response. This is an **uncertain outcome**, not a failed transaction. Preserve the original decision; recognize retries and handle subsequent network advice, reversal, or expiry according to applicable rules.

Never reverse automatically merely because a response timed out. The network and merchant may have observed a different outcome.

Failure C: Fraud service timeout

Apply the explicit risk-approved degraded policy. Do not default to approving every transaction. Use bounded retries and circuit-breaker behavior where suitable.

Failure D: Authoritative database unavailable

The proposed normal online path cannot safely approve without durably recording the reservation. Return the defined failure response or invoke a separately governed network-supported contingency arrangement.

Failure E: Event broker unavailable

```
Authorization DB
      |
      v
Durable Outbox
      |
      X  Broker unavailable

Outbox event remains for retry.
```

The committed financial decision remains valid. The event publisher retries later, and consumers must handle duplicates.

Summary

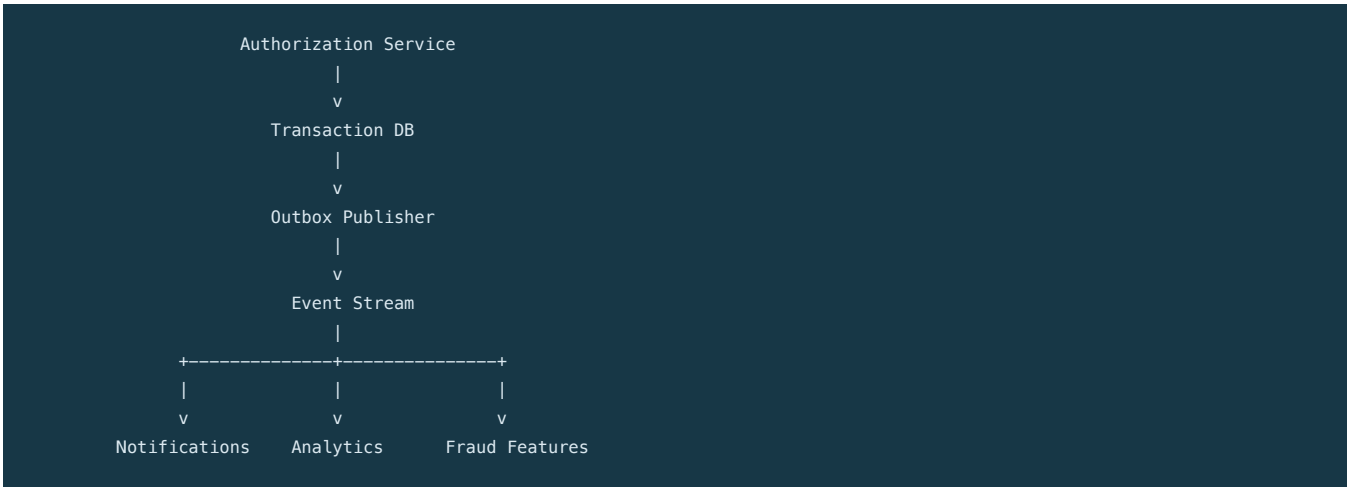
Failure	Correct response
Duplicate authorization	Recover previous decision
Crash before commit	Roll back; permit safe retry
Commit then lost response	Keep original decision; reconcile uncertain outcome
Fraud timeout	Apply defined risk degradation
Database unavailable	Avoid unsupported approval
Broker unavailable	Retain event in durable outbox
Duplicate reversal	Release only once
Late clearing	Match and apply lifecycle rules
Stale account state	Revalidate before reservation
Replayed event	Idempotent downstream processing

CHAPTER

16. Where Kafka or a message broker belongs

Do not insert an asynchronous event broker into the synchronous decision path without a business and protocol reason. The card network needs an immediate authorization response.

Use an event stream **after the durable decision** for downstream workloads:



Benefits: independent consumers, reduced coupling, replay when supported, and protection of critical response latency.

Queue vs. stream

Requirement	Traditional queue	Kafka-style event stream
Background jobs	Strong fit	Possible
Multiple independent subscribers	Requires suitable pub/sub	Consumer groups fit naturally
Historical replay	Depends on queue design	Strong fit
Per-key ordering	Product-specific	Per-partition ordering
Simple work dispatch	Often easier	Often more infrastructure

For downstream account-level processing, partition events by a stable account key when per-account ordering is required. Add sequence numbers or versions so old/repeated events cannot corrupt projections.

Do not mistake Kafka for an authoritative credit ledger or reservation database.

Concept — Transactional outbox: Write business state and an event record in one local database transaction, then asynchronously publish the event. See AWS Prescriptive Guidance.

CHAPTER

17. High availability and multi-region design

Separate stateless services from authoritative state:

Easier to replicate	Requires stricter coordination
Authorization workers	Account credit state
Protocol workers (subject to network session rules)	Authorization decisions and holds
Routing and some policy evaluation	Durable request identity
Risk inference instances	Key-management services and outbox

Step 1: Multiple availability zones



A highly available API alone is insufficient; its authoritative database must also survive expected component failures.

Replication trade-offs

Strategy	Advantage	Risk / cost
Synchronous durable replication	Stronger durability before acknowledgement	Higher latency/coordination cost
Asynchronous replication	Faster acknowledgment	Recently committed writes may be missing after failover

Choose based on tested durability, acknowledgement, quorum, failover, and recovery-point guarantees—not the marketing label of the database.

Why active-active financial writes are difficult



If both independently write against stale replicas, total approvals can exceed available credit.

Initial choice: Single authoritative writer per account





Concept — Fencing: Ownership epochs or comparable mechanisms prevent an old primary from writing after ownership has moved to a new primary.

A global strongly consistent database is an alternative, but may increase cross-region latency and coordination complexity. Use it if requirements justify the cost.

Disaster recovery targets

- **RPO:** Maximum acceptable data loss.
- **RTO:** Maximum acceptable recovery duration.

Recovery must verify not only that the database is reachable but also that previously acknowledged authorization decisions, holds, and idempotency records remain correct.

Prevent **split brain**, where two regions simultaneously believe they own the same account's writes.

CHAPTER

18. Scale authorization capacity

Using the hypothetical ~5,800 TPS peak, possible bottlenecks include protocol connections, HSM capacity, feature lookups, inference, database transactions, account-row contention, and network hops.

Component	Scaling approach	Why
Network gateways	Multiple approved endpoints / connections	Ingress and resilience
Authorization workers	Horizontal scaling	Largely stateless
Risk inference	Multiple serving instances	Parallel model evaluation
Fraud feature store	Distributed key-based access	Low-latency lookups
Authoritative database	Tune, partition, then shard as required	Transactional capacity
HSM services	Approved redundant capacity	Cryptographic availability
Event consumers	Partition-aware consumer scaling	Downstream throughput

Shard by the correct business identity

Why `account_id`? Multiple cards may share the same credit account. Sharding by card could split the authoritative spending capacity across nodes.

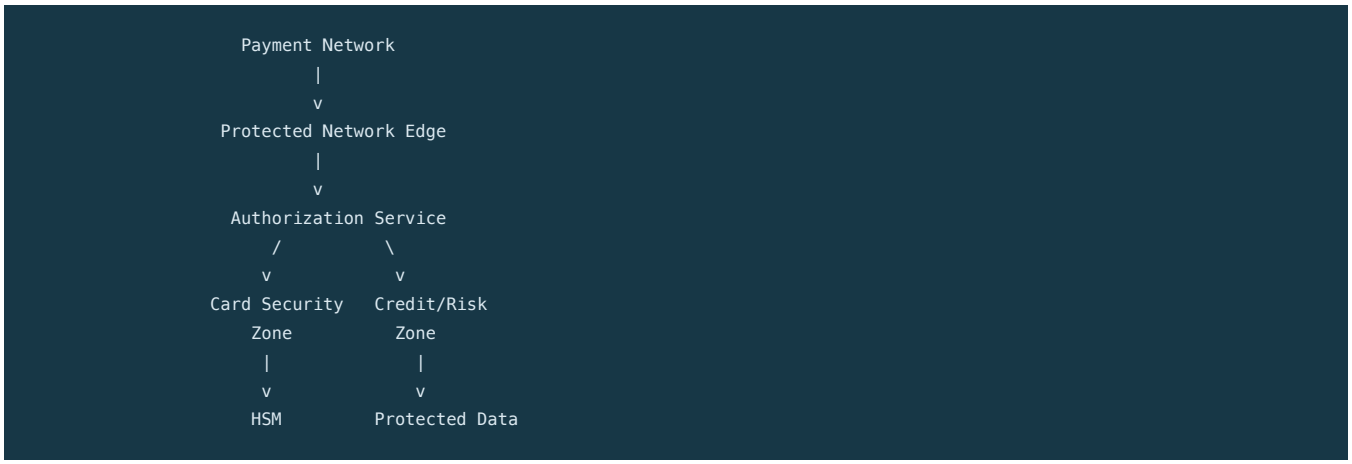


A real shard map would use a managed mapping, partitioning, hashing, or ranges—rather than literal alphabetical slices.

Trade-offs: routing, rebalancing, hotspot handling, difficult cross-account queries, and safe migration. Start with a simpler database until a measured limit justifies sharding.

CHAPTER

19. Security architecture



Concern	Architectural response
Network identity	Authenticated/protected connections
Card credentials	Isolation from general business services
Cryptographic keys	HSM-backed operations and controls
Data at rest	Classification, encryption, minimization
Service access	Least privilege and workload identities
Internal links	Protected service-to-service communication
Access records	Tamper-resistant audit trails
Operations	Privileged access controls and separation of duties
Logs	Redaction of credentials and sensitive auth data

PCI DSS

PCI DSS governs protection of payment account data for in-scope entities and systems. See the PCI SSC official document library for PCI DSS v4.0.1.

Sensitive authentication data has special post-authorization retention restrictions. Issuer-specific exceptions must not be mistaken for general permission to store sensitive data.

Tokenization

Tokenization: Replacing a sensitive payment identifier with a substitute identifier to reduce exposure. A generic internal reference token and an EMV payment token serve different purposes and have different controls.

Business services should use internal card/account references when feasible instead of handling raw primary account numbers.

CHAPTER

20. Observability and operations

Metric	Detects
p50/p95/p99 response latency	Slow critical-path behavior
Technical error rate	Processing reliability
Approval/decline reason distribution	Unusual policy or business changes
Database commit time	Persistence bottlenecks
Row-lock contention	Hot accounts or long transactions
HSM errors and latency	Cryptographic bottlenecks
Fraud service timeouts	Dependency failure
Duplicate request rate	Network retry behavior
Outbox backlog	Downstream delivery problems
Reversal match failures	Lifecycle errors
Reconciliation mismatches	Divergence of financial state

A high **business decline rate** is not necessarily a **technical error rate**. Report them separately.

Tracing

```
Network Request
      |
      v
Correlation ID
      |
      +--> Credential check
      +--> Risk decision
      +--> Credit reservation
      +--> Response
```

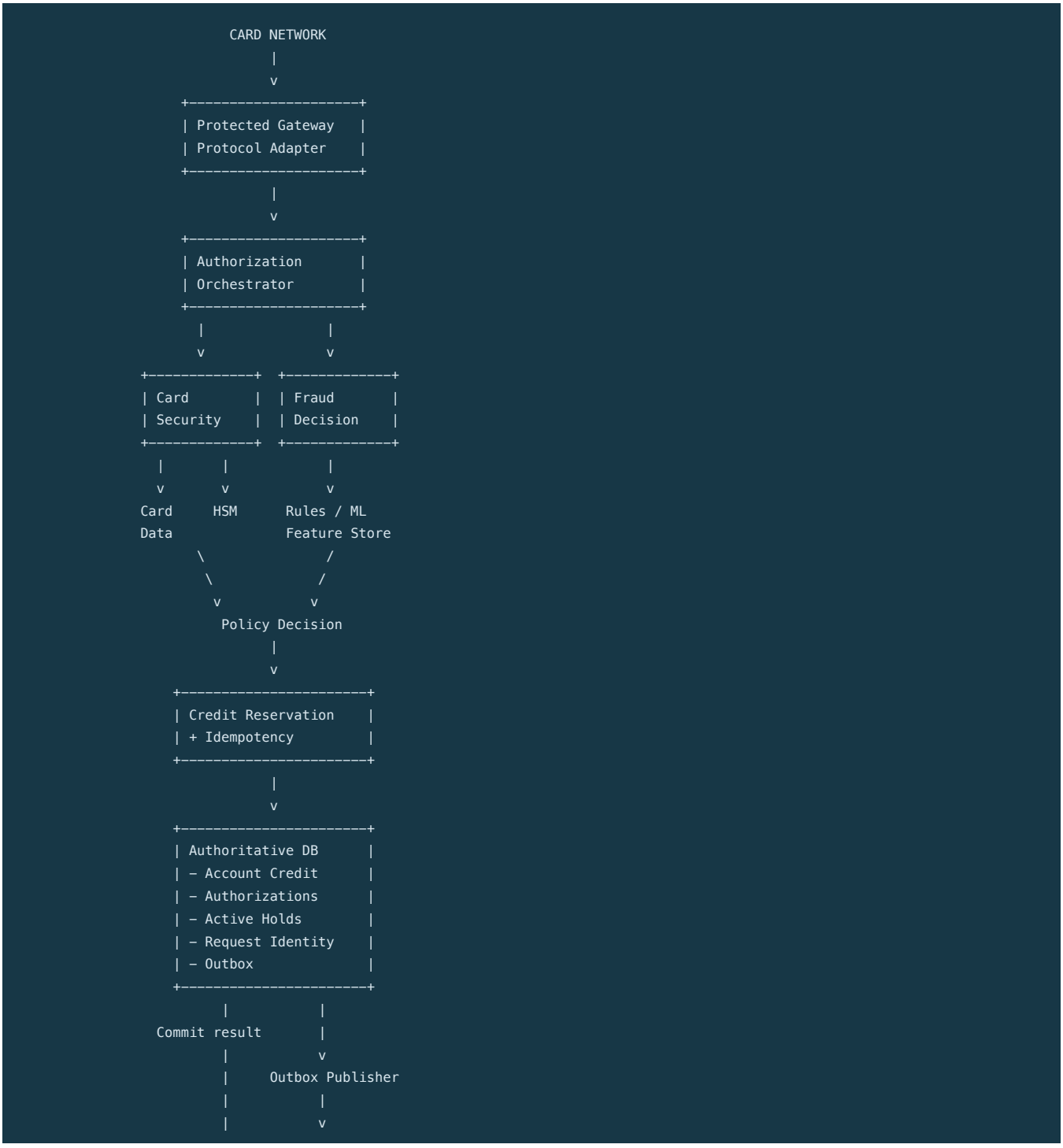
Propagate a correlation identity without logging protected payment credentials.

Operational readiness

Define deployment and rollback processes, load testing, capacity planning, backups, restore verification, failure injection, recovery exercises, incident management, security response, runbooks, and service ownership.

CHAPTER

21. Final reference architecture



```
                        |       Event Broker
                        |           |
                        |      +----+-----------+
                        |      |        |       |
                        |      v        v       v
                        |   Alerts   Analytics  Fraud
                        |                       Updates
                        v
                  Response Adapter
                        |
                        v
                   CARD NETWORK
```

Clearing, posting, ledger, and settlement are separate components and processes. They share stable authorization references and reconcile exposure correctly.

CHAPTER

22. Decisions and rejected alternatives

Decision	Rationale	Alternative / when to reconsider
Protocol adapter	Isolates network-specific formats	Direct protocol model in services
Synchronous authorization	Network requires prompt decision	Async processing for noncritical work
Transactional authoritative DB	Atomic reservations and unique constraints	Distributed transactional stores
Atomic conditional reservation	Prevents double-use of credit	Explicit locking or optimistic versioning
Durable request identity	Prevents duplicate holds	Cache only as optimization
Mandatory synchronous risk checks	Prevents inappropriate pre-approval	Async risk for post-decision analytics
Online fraud features	Fast inference data	Analytical query store for offline use
Transactional outbox	Avoids DB/broker dual-write gap	Direct publish is unsafe without equivalent guarantees
Asynchronous analytics and alerts	Protects approval latency	Synchronous calls only when required
Account-level partitioning	Keeps shared credit state together	Card-based sharding risks splitting shared line
Single-writer account ownership	Prevents conflicting cross-region writes	Globally coordinated multi-writer storage
Card-security isolation	Limits exposure of credentials and keys	Broad application access is inappropriate

CHAPTER

23. Variations that change the architecture

A. Much higher transaction volume

Scale stateless services and model inference; then measure database saturation and contention. Partition and shard when there is a proven bottleneck.

B. Multi-region active-active

Use explicit account ownership and safe failover, globally coordinated writes, or a carefully bounded/reconciled allocation strategy. Eventual replica synchronization alone does not enforce a global spending limit.

C. Real-time velocity limits

Use stream-derived features for probabilistic risk. Enforce hard transaction-count limits atomically or with correctly coordinated state.

D. Hotel / rental preauthorization

Handle estimated amounts, incremental authorization, partial reversal, and amounts that differ at clearing.

E. Network outage

The issuer cannot perform normal online authorization without the relevant messages. Any stand-in or offline authorization requires a separately governed network-supported mechanism.

F. Multi-currency

Define reserve currency, rate source, rounding, FX adjustments, and the relationship between authorization and later clearing amounts.

G. Charge cards / dynamic spending power

A simple fixed credit-limit formula may not apply. Replace it with the product-specific spending-capacity policy while preserving durable, auditable decisions and controlled concurrency.

CHAPTER

24. Summary: Key concepts demonstrated

Concept	Principle
Payment lifecycle	Authorization is not clearing or settlement
Atomic transactions	Related financial changes commit together
Concurrency	Prevent simultaneous requests from spending the same available credit
Idempotency	Retry the same logical operation without repeating the financial effect
Uncertain outcomes	A timeout is not proof of non-commit
Fraud architecture	Mandatory checks belong before approval
Authoritative state vs. projections	Financial decisions require the correct source of truth
Transactional outbox	Couple business commit with durable intent to publish
Partitioning	Choose keys based on shared business invariants
Multi-region consistency	Availability cannot rely on uncontrolled conflicting writes

Security	Protect credentials, keys, and access boundaries
Reliability	Define behavior under dependency failures

Interview-ready explanation

A credit card authorization system receives financial messages from a payment network and returns a low-latency decision using card validity, risk, account policy, and available spending capacity. The core design uses a transactional source of truth for both authorization decisions and credit reservations. Atomic updates prevent concurrent authorizations from incorrectly consuming the same available credit. Durable idempotency prevents duplicate network requests from creating duplicate holds.

Required credential and risk decisions occur before approval, while analytics, notifications, and other noncritical work are decoupled through durable asynchronous events. The system handles reversals, expiry, clearing reconciliation, and uncertain network outcomes. Multi-AZ infrastructure, carefully controlled regional ownership, secure cryptographic boundaries, and tested recovery procedures increase resilience without weakening financial invariants.

Next design: Double-Entry Financial Ledger — immutable journal entries, debit/credit rules, balance derivation, posting, reconciliation, concurrency, and recovery.

CHAPTER

Official references

1. American Express — U.S. Merchant Reference Guide — authorization, reversals, submission, and settlement.
2. AWS Prescriptive Guidance — Transactional outbox pattern — database/event dual-write risk, duplicate delivery, and recovery.
3. AWS Prescriptive Guidance — Cloud design patterns — anti-corruption layers, retries, circuit breakers, and workflows.
4. PCI Security Standards Council — Document library — PCI DSS v4.0.1 and account-data protection requirements.